ML Model Deployment Strategies and Model Serving using FastAPI

Dataset: Titanic - Machine Learning from Disaster (Kaggle), train.csv

Summary of results

A Random Forest pipeline (median imputation and scaling for numeric features, most-frequent imputation and one-hot encoding for categorical features) was trained on 80% of train.csv (712 rows) and reached 0.8101 accuracy on the 20% hold-out (179 rows). The complete pipeline was saved once as model.pkl and reused for online inference (1 record), batch inference (20 records, 17/20 matched the actual labels), streaming inference (10 records, 1 per second) and FastAPI serving (/health and /predict with Pydantic validation). All inference records come from the unseen hold-out set.

Task 6: Observations

1. How does Online Inference differ from Batch Inference?

Online inference scores one record at a time, on demand, and returns the result immediately, so latency matters most (e.g. a user submits a form and gets an answer). Batch inference scores many records together in a single call, usually on a schedule or on stored data, and the results are saved for later use (here, 20 passengers written to batch_predictions.csv). Batch is more efficient per record because the work is vectorised, but results are not instant.

2. What is the difference between Streaming Inference and Batch Inference?

Batch inference works on a fixed, finite set of records that already exist and processes them all at once. Streaming inference handles records as they arrive continuously, one at a time (or in micro-batches), producing a prediction for each event right away. In Task 4 the 10 records were processed one per second. Streaming gives low-delay results on live data; batch gives high throughput on historical data.

3. Why is FastAPI used for model serving?

FastAPI is fast (async, built on Starlette/Uvicorn), simple to write in plain Python, and validates input automatically using Pydantic type hints, rejecting bad requests with clear 422 errors. It generates interactive Swagger UI documentation at /docs, so the API can be tested without extra tools. It makes it easy to expose the saved model as a REST service that any client or language can call.

4. Why should the same trained model be reused for all inference strategies?

Using one saved pipeline (model.pkl) guarantees consistent predictions: the same record gets the same answer whether it is scored online, in batch, in a stream or through the API. It also keeps preprocessing identical between training and serving, which avoids training-serving skew, and avoids wasted retraining. It makes results reproducible, easier to version and audit, and means a model update only has to be deployed in one place.